Real-Time Hand Gesture Tracking and Interactive Drawing System

1.1 Introduction:

1.1.1 Purpose of the System:

The purpose of this document is to define the Software Requirements Specification (SRS) for a web application that allows users to draw on a canvas using their index and middle fingers. The application includes features like selecting drawing colors, shapes, brush sizes, and other functionalities similar to a standard paint application. The system integrates a custom-trained machine learning model to track finger movements without relying on external libraries or frameworks like YOLO, OpenCV, or MediaPipe

1.1.2 Scope of the System:

The application provides a drawing interface powered by a custom-trained machine learning model for finger tracking. It is designed to:

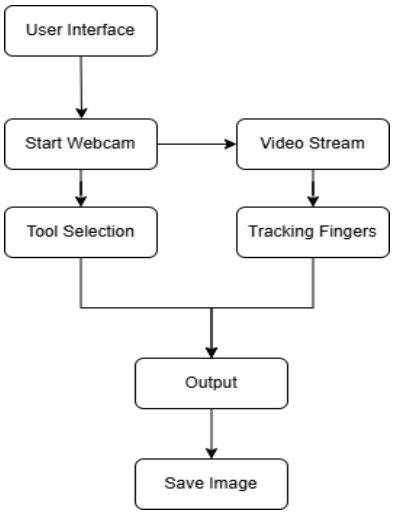
- Enable real-time finger tracking using a proprietary algorithm and model.
- Allow users to draw freehand, create shapes, and select brush sizes and colors.
- Be accessible via a web browser, with the backend built using Python for machine learning and Node.js with Express.js for server-side functionality.

1.2 General Description of the System:

1.2.1 Overall Description:

The application provides a drawing interface powered by a custom-trained machine learning model for finger tracking. It is designed to:

- Enable real-time finger tracking using a proprietary algorithm and model.
- Allow users to draw freehand, create shapes, and select brush sizes and colors.
- Be accessible via a web browser, with the backend built using Python for machine learning and Node.js with Express.js for server-side functionality.



1.2.2 Feasibility Study:

1. **Technical Feasibility:**
   - Uses modern technologies like Python and Node.js.
   - Runs on devices with camera support and modern web browsers.
   - Custom ML models ensure flexibility and independence from external libraries.

2. **Economic Feasibility:**
   - Utilizes open-source tools and technologies to minimize costs.
   - Major expenses include development time and server hosting.

3. **Operational Feasibility:**
   - User-friendly system designed for easy accessibility.
   - Targets a wide audience, including artists, educators, and casual users.

1.3 Functional Requirements:

1.3.1 Module Description:

Video Capture Module
- Captures live video feed from the webcam.
- Adjusts resolution and frame rate for optimal performance.
- Passes frames to processing modules.

Hand Detection and Tracking Module
- Detects and tracks hand movements in real time.
- Identifies key points like fingertips and palm position.
- Provides coordinates for further processing.

Gesture Recognition Module
- Recognizes gestures such as index finger up, fist, or open palm.
- Maps gestures to actions like drawing, erasing, or shape selection.
- Uses custom ML models or heuristic-based classification.

Drawing Module
- Allows users to draw freehand and create shapes.
- Supports brush size, color selection, and stroke thickness.
- Provides undo/redo functionality.

User Interface (UI) Module
- Displays video feed with gesture-based controls.
- Provides buttons for tools like brush, eraser, and shape selector.
- Offers a settings menu for user preferences.

Event Handling Module
- Detects keyboard shortcuts and system events.
- Manages transitions like drawing mode, erasing mode, and clearing canvas.

Custom Algorithm Module

- Implements hand segmentation, contour analysis, and feature extraction.
- Uses in-house algorithms for gesture classification.

Data Management and Analytics Module

- Logs hand positions and gestures for debugging.
- Provides performance metrics and analytics for accuracy improvement.

Save and Export Module

- Saves drawings in PNG, JPEG, or SVG formats.
- Supports exporting drawings with transparent backgrounds.

Undo/Redo Module

- Allows users to undo or redo recent actions.
- Maintains drawing history for better flexibility.

System Optimization and Performance Module

- Ensures real-time gesture recognition with minimal lag.
- Reduces computational overhead for smoother performance.

1.3.2 Functions of Various Users of the System:

End Users

- Interact with the system to draw on the canvas using hand gestures.
- Customize drawings using brush sizes, colors, and shape selection.
- Save or export completed drawings for future use.

Developers

- Enhance gesture recognition accuracy by improving the ML model.
- Add new features and optimize application performance.
- Ensure compatibility across different devices and browsers.

System Administrators

- Maintain backend servers and system performance.
- Manage updates to the machine learning model and other components.
- Address user-reported issues and system failures.

1.4.Non-Functional Requirements:

Performance:

- The system should provide real-time hand tracking with minimal latency.
- Gesture recognition should have a response time of less than 100ms.
- The application should render drawings smoothly without noticeable lag.

Scalability:

- The backend should support multiple concurrent users without performance degradation.

- The system should allow for future expansion, such as adding multi-hand tracking or new gestures.
- Should be able to integrate with AR/VR environments in the future.

Usability:

- The user interface should be intuitive and easy to use without requiring extensive training.
- Users should receive real-time visual feedback for gestures and interactions.
- The system should provide help documentation or tooltips for first-time users.

Security:

- The system should not store user gesture data or drawings without explicit consent.
- All user interactions should be processed locally or securely on the server.
- The application should follow standard web security protocols to prevent unauthorized access.

Compatibility:

- The application should work on modern web browsers like Chrome, Firefox, and Edge.
- The system should support devices with different screen sizes, including PCs, tablets, and smartphones.
- The software should be operating system-independent and work on Windows, macOS, and Linux.

Reliability:

- The system should have 99.5% uptime, ensuring availability for users.
- In case of failure, it should recover quickly without data loss.
- The hand tracking and gesture recognition should work in varying lighting conditions with at least 90% accuracy.

Maintainability:

- The codebase should be modular and well-documented to allow for easy updates.
- Developers should be able to replace or upgrade individual modules without affecting the entire system.
- System logs should be maintained for troubleshooting and performance monitoring.

Portability:

- The application should be deployable on multiple platforms, including local machines and cloud environments.
- The system should allow easy migration between different hardware and software setups.

1.5Interface Requirements:

1.5.1 GUI

- The application provides a responsive and intuitive user interface.

- The canvas is the main interaction area, occupying the majority of the screen.
- Tools such as the brush, eraser, shape selector, and color palette are available as buttons or dropdowns around the canvas.
- A real-time preview of selected brush size and color is displayed.
- Gesture-based controls are visually indicated on the interface.
- A settings menu is available for configuring preferences such as brush sensitivity or gesture calibration.

1.5.2 Hardware Interface

- A camera-enabled device (e.g., laptop, smartphone, or tablet) is required for finger tracking.
- Devices must support HD video input for optimal tracking performance.
- Minimum hardware specifications:
    - Processor: Dual-core CPU, 2.0 GHz or higher
    - RAM: 4 GB or higher
    - Camera resolution: 720p or higher

1.5.3 Software Interface

- The application is compatible with modern web browsers supporting HTML5, JavaScript, and APIs for accessing camera feeds.
- The backend interacts with:
    - Custom ML Model: Implemented in Python, deployed on the server, and integrated via APIs.
    - Node.js Server: Handles client requests, communicates with the ML model, and manages real-time tracking data.
- The frontend is built with HTML, CSS, and JavaScript frameworks.
- Third-party libraries or APIs (if any) must comply with standard web security protocols.

1.6 Data Dictionary:

1. Table Name : Login_Table

Description: Stores login credentials for Users.

Field Name	Data Type	Description	Constraints
user_id	Integer	Unique identifier for each user	Primary Key, Auto Inc
password	String	Password of the account	6-12 characters
username	String	Username of the user	Unique, Max 50 characters
email	String	Email address of the user	Unique, Valid Email Format
role	String	Role of the user (End User, Admin,	Predefined Values

Field Name	Data Type	Description	Constraints
		Developer)	
created_at	Timestamp	Account creation date and time	Auto-generated

2. Table Name : FingerTracking_Table

Description: Stores the details of fingers positioning

Field Name	Data Type	Description	Constraints
tracking_id	Integer	Unique identifier for each tracking event	Primary Key, Auto Increment
user_id	Integer	Identifier of the user being tracked	Foreign Key (User Table)
x_position	Float	X-coordinate of the finger position	Range: 0 - Canvas Width
y_position	Float	Y-coordinate of the finger position	Range: 0 - Canvas Height

.

3. Table Name : Training_Data

Description: Stores the details of different Samples.

Field Name	Data Type	Description	Constraints
sample_id	Integer	Unique identifier for each training sample	Primary Key, Auto Increment
frame_data	JSON	Stores the raw image/frame data used for training	Base64 or Image File Path
label	String	Label indicating the position or gesture of the fingers	Predefined Values (e.g., Index Up, Middle Up, Both Up)
timestamp	Timestamp	Time when the sample was collected	Auto-generated

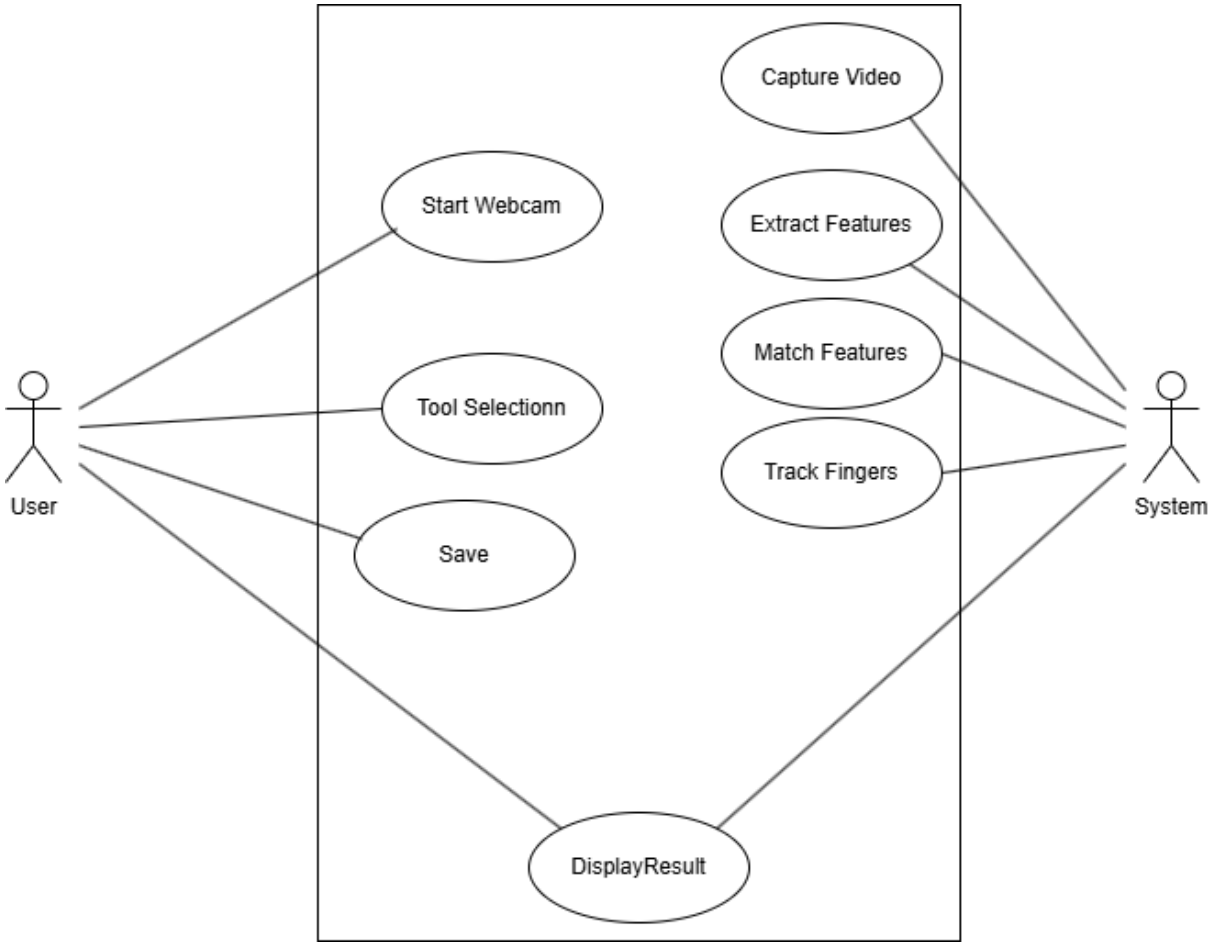
3. Table Name : Training_Data

Description: Stores the metadata of the models.

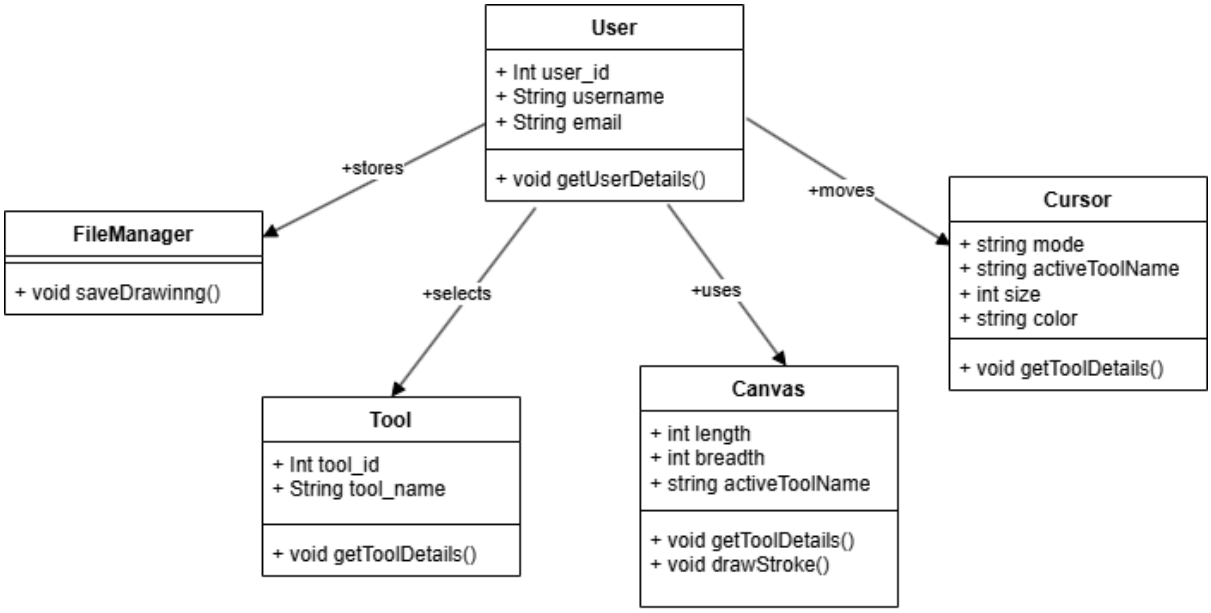
Field Name	Data Type	Description	Constraints
model_id	Integer	Unique identifier for each trained model	Primary Key, Auto Increment
model_name	String	Name assigned to the trained model	Max 100 characters

Field Name	Data Type	Description	Constraints
version	String	Version number of the model	Semantic Versioning (e.g., 1.0.0)
accuracy	Float	Accuracy score of the trained model	Range: 0.0 - 1.0
training_date	Timestamp	Date when the model was trained	Auto-generated

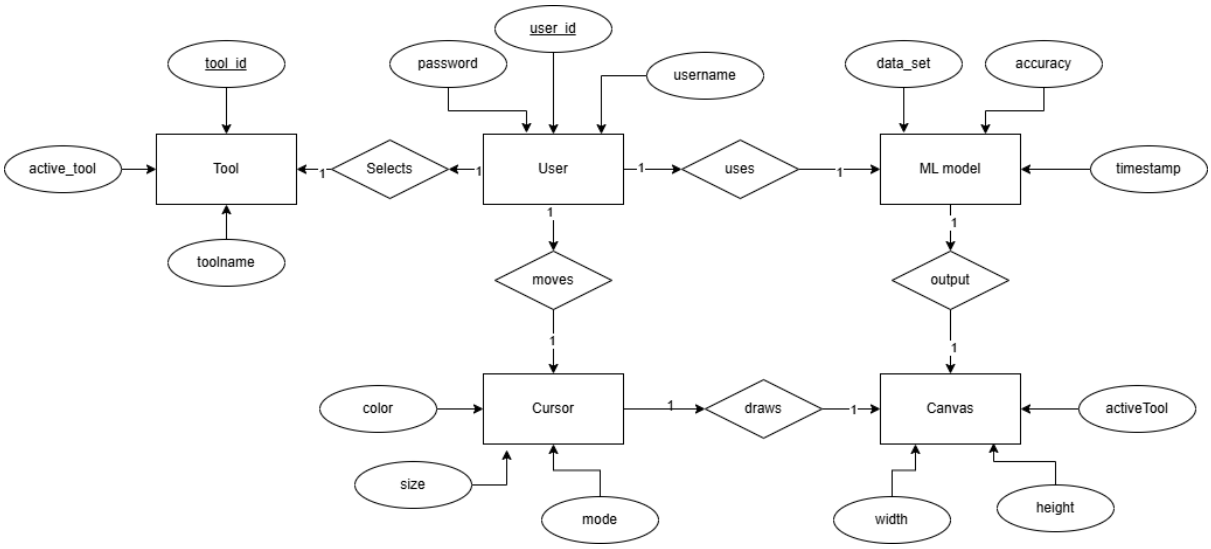
Use Case Diagram of the System:



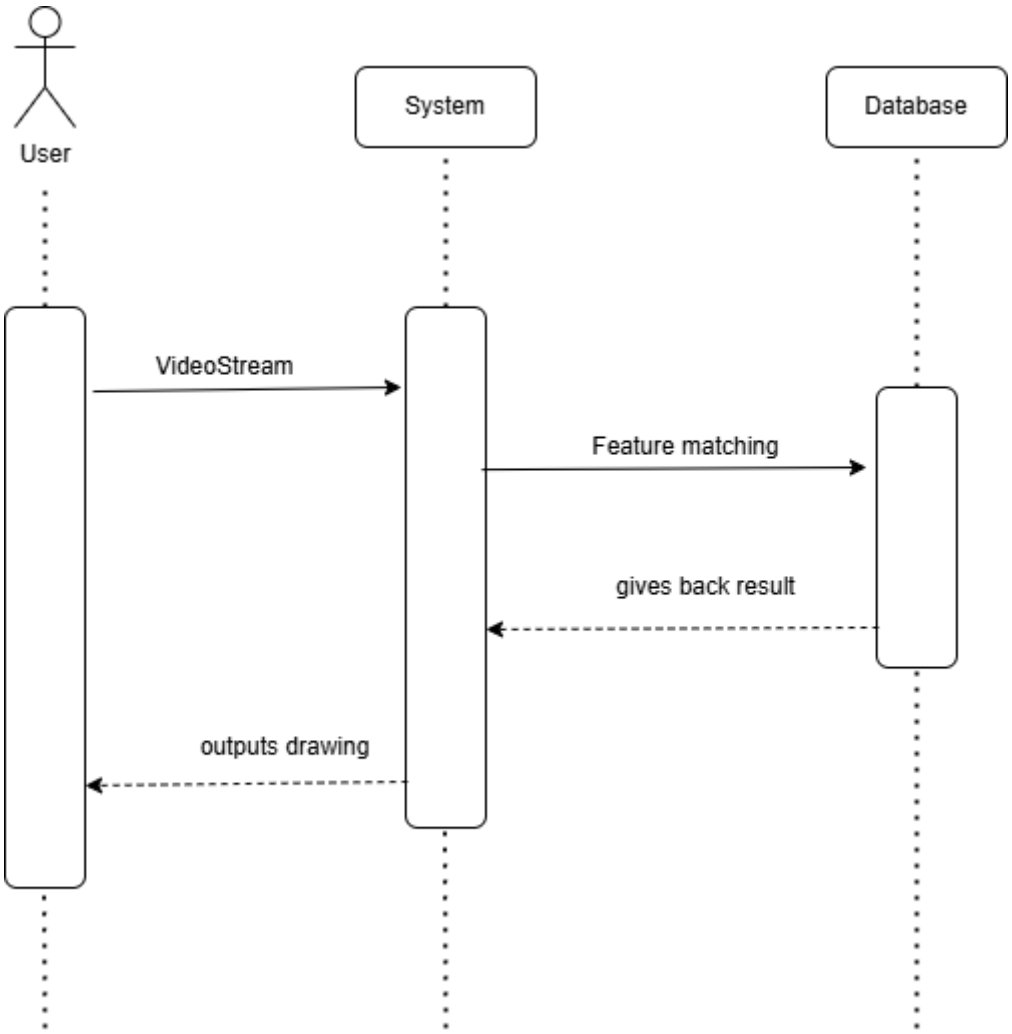
Class Diagram of the System:



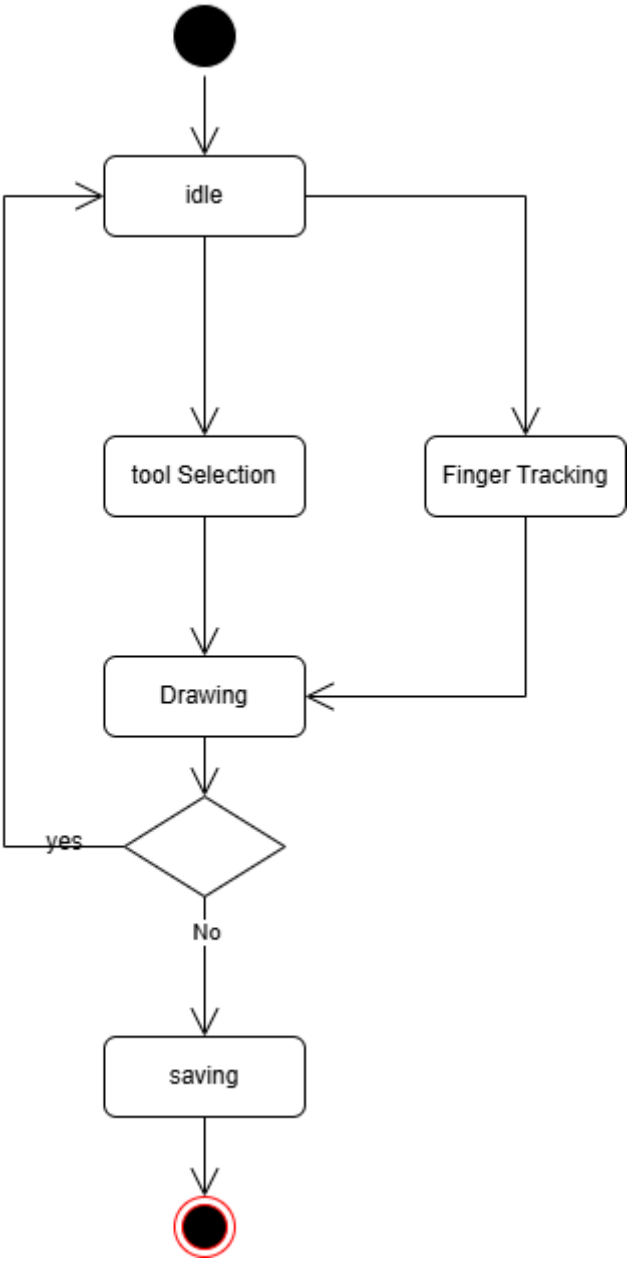
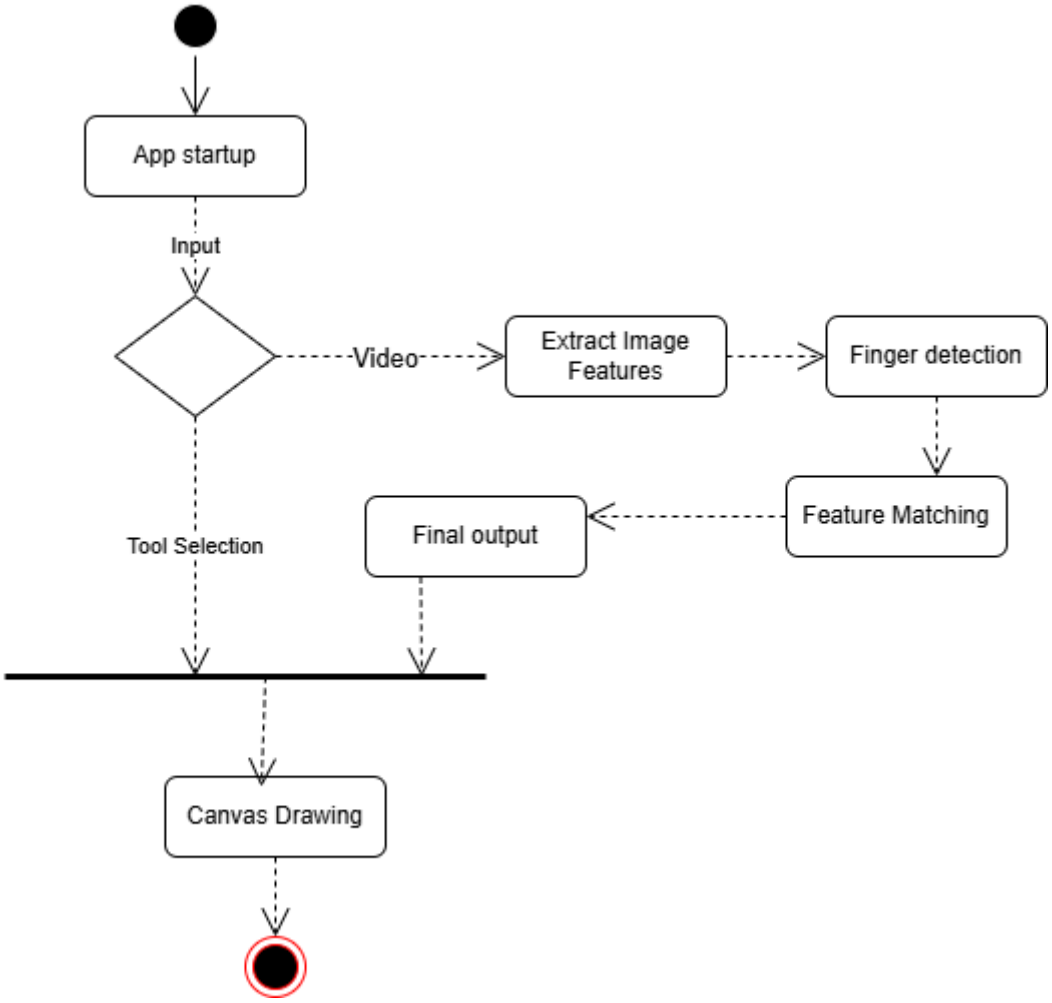
ER Diagram of the System:



Sequence Diagram of the System:



State Diagram of the System:



Activity Diagram of the System: